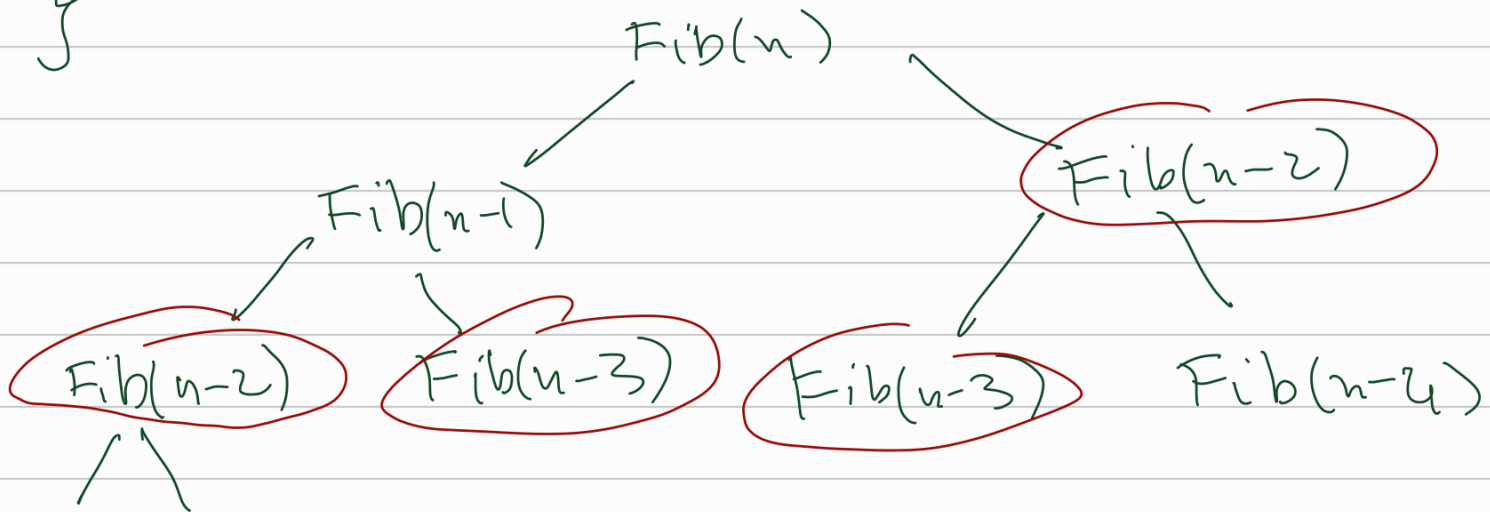


Dynamic Programming:

0	1	2	3	5	8	13	21	34
1	2	3	4	5	6	7	8	9

Fib(n)

```
{  
  if (n == 1) return 0;  
  if (n == 2) return 1;  
  return Fib(n-1) + Fib(n-2);  
}
```

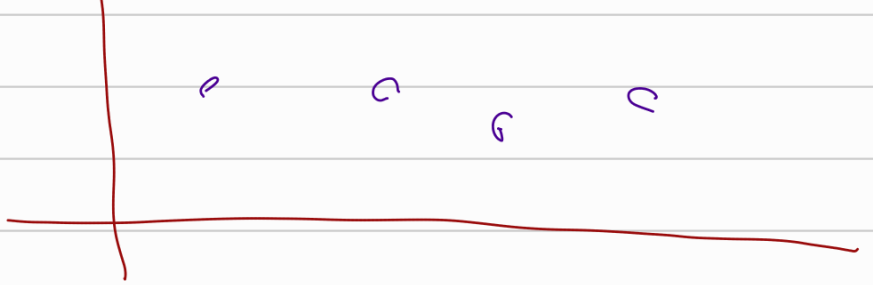


$O(2^n)$, $O(1.5^n)$

exponential

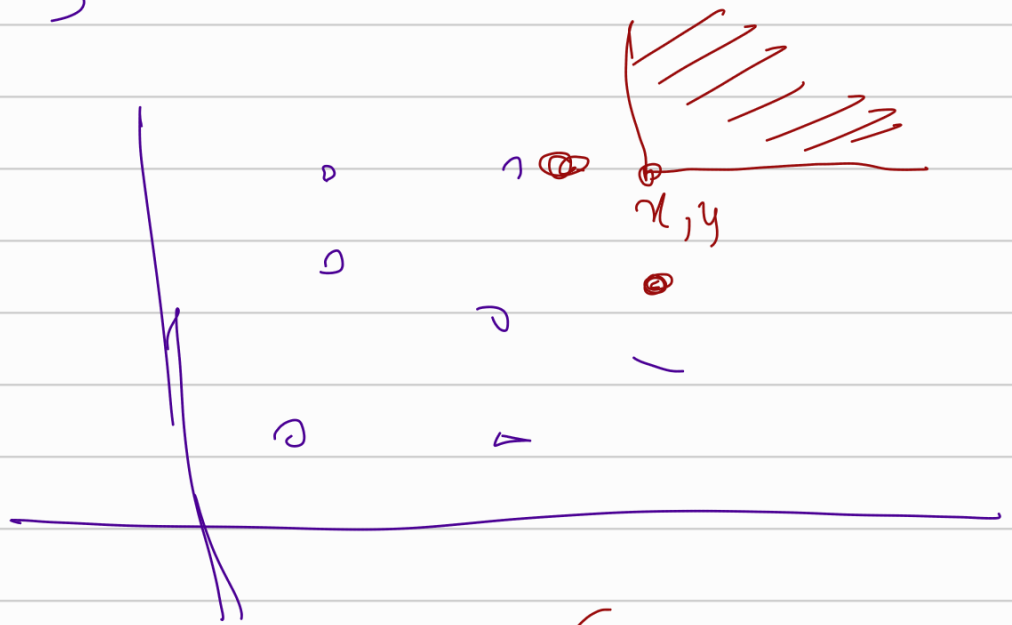
multiple
subproblems

(n, y)



$$\text{sol}(x, y) = \text{maxCoins}(x, y)$$

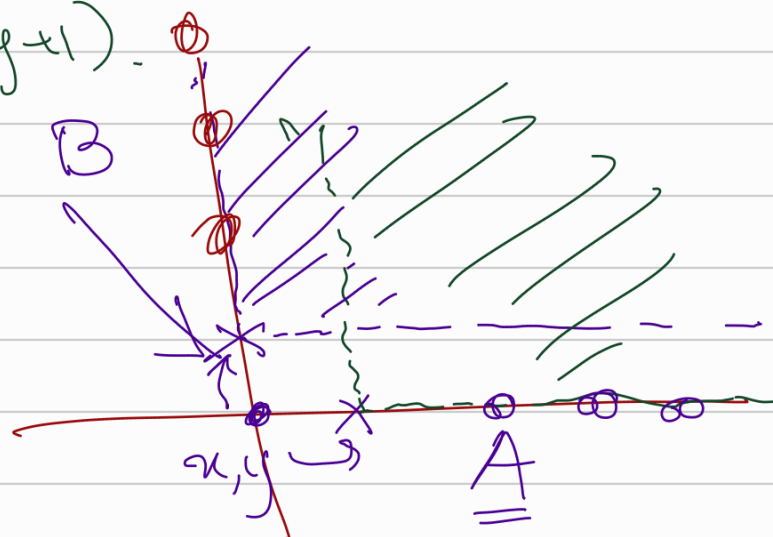
max Coins that can be picked
 from, $x \rightarrow$
 $y \uparrow$



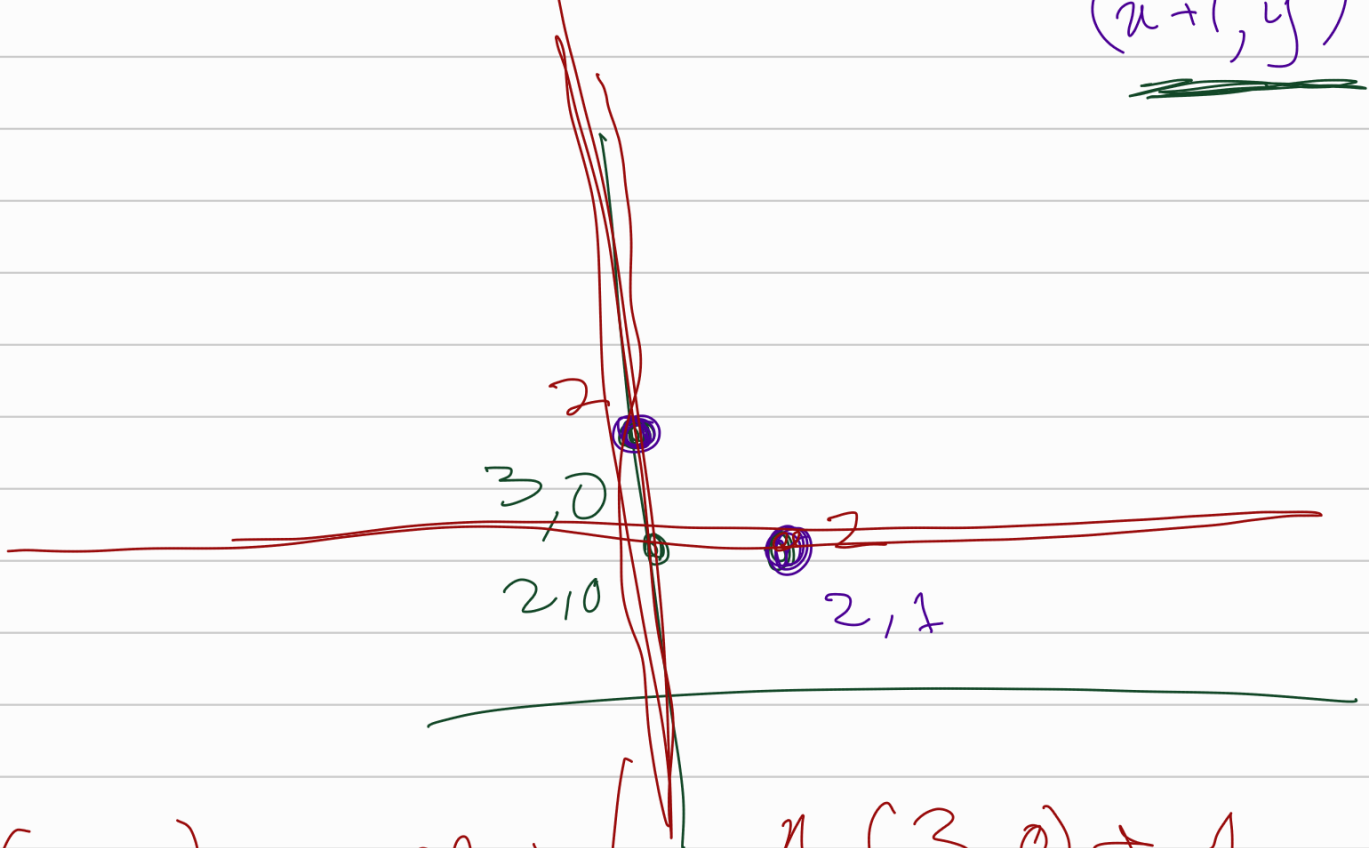
$$\text{maxCoin}(x, y) = \max \left(\begin{array}{l} 1/0 + \text{maxCoins}(x+1, y) \\ 1/0 + \text{maxCoin}(x, y+1) \end{array} \right)$$

$$1/0 + \text{maxCoin}(x, y+1)$$

$$(x, y+1) = B$$



$(x+1, y)$



$$A(2,0) = \max \left(A(3,0) + 1, \right. \\ \left. A(2,1) + 1 \right) \\ = \max(3, 3) = 3$$

- ① Design a more efficient algorithm to find a solution to Coldex.

$O(n^2)$

$O(n \log n)$

$O(n)$

$\Rightarrow \underline{A}$

- ② Prove that finding solution to higher dimension version of this problem (d) is NP-Hard.

$\rightarrow A$